Document number: P0356R0
Date: 2016-05-22
Project: Programming Language C++, Library Evolution Working Group
Reply-to: *Tomasz Kamiński <tomaszkam at gmail dot com>*

Simplified partial function application

1. Introduction

This document proposes and introduction of the new library functions for performing partial function application and act as replacement for existing `std::bind`.

This paper addresses LEWG Bug 40: variadic bind.

2. History

This proposal is successor of the N4171: Parameter group placeholders for bind, that was proposing an extension of the existing `std::bind` to introduce new class of placeholders that would presents group of call arguments instead of one.

In this paper two new `bind_front` and `bind_back` are proposed that allow user to provide values that will be passed as first or last arguments to stored callable. The author believes that this solution is in-line with LEWG recommendation for the original paper, that suggested to introduce only `_all` placeholder.

3. Motivation and Scope

This paper proposes two new functions for partial function application:

- `bind_front` for binding values of first arguments
- `bind_back` for binding values of last arguments

In other worlds `bind_front(f, bound_args...)(call_args...)` is equivalent to `std::invoke(f, bound_args..., call_args...)` and `bind_back(f, bound_args...)(call_args...)` is equivalent to `std::invoke(f, call_args..., bound_args...)`.

It is worth to notice that proposed functions provide both superset of existing `std::bind` functionality: their support passing variable number of arguments, but does not allow arbitrary reordering or removal of the arguments. However author believes that proposed simplified functionality covers most of use cases for original `std::bind`.

3.1. Passing arguments

Let consider an example task of writing the functor that will invoke `process` method on copy of `strategy` object:

```
struct Strategy { double process(std:string, std::string, double, double); };
std::unique_ptr<Strategy> createStrategy();
```

Firstly, such functor should not cause any additional overhead caused by passing the argument values form the call side to the stored callable. To achieve desired effect in case of lambda based solution, we can use forwarding reference in combination with variadic number of arguments:

```
[s = createStrategy()] (auto&&... args) { return s->process(std::forward<decltype(args)>(args)...); }
```

In case of the functors produced by `std::bind`, perfect forwarding is used by default for the all call arguments that are passed in place of placeholders, so same effect may be achieved using:

```
std::bind(&Strategy::process, createStrategy(), _1, _2, _3, _4)
```

However use of the named placeholders has is own drawbacks. Firstly change to the functor code is required each time when the number of arguments is changed. Secondly, it allows user to write a code that will pass same value to the function multiple times, by using same placeholder twice, which in case of use of move semantics may lead to passing of unspecified values as arguments. For example values of first and second argument are unspecified in case of following invocation:

```
auto f = std::bind(&Strategy::process, createStrategy(), _1, _1, _2, _2);
f(std::string("some_string"), 1);
```

In contrast in case of proposed `bind_front`/`bind_back` function, all arguments provided on the call side are forwarded to the callable. As consequence the user is not required to manually write boilerplate code for perfect forwarding nor are exposed to potential errors caused by use of placeholders:

```
bind_front(&Strategy::process, createStrategy())
```

3.2. Propagating mutability

In our previous example the strategy object was stored in the callable indirectly by the use of the smart pointer, so it mutability was not affected by the functor. However in case of storing object by value we would like to propagate constness from the functor. That means for each of the following declarations:

```
auto f = [s = Strategy{}] (auto&&... args) { return s.process(std::forward<decltype(args)>(args)...); }; // 1
```

```
auto f = std::bind(&Strategy::process, Strategy{}, _1, _2, _3, _4); // 2
auto f = bind_front(&Strategy::process, Strategy{}); // 3
```

Invocation on mutable version of the functor (`f`) shall invoke `process` method on mutable object (call well-formed), however in case of const qualified one (`std::as_const(f)`) `process` method shall be invoked on const object (call ill-formed). This functionality is supported both by existing `std::bind` (2) and proposed `bind_front`/`bind_back` (3), however it is not in case of the lambda (1). This is caused by the fact that closure created by the lambda has only one overload of the call operator that is const qualified by default.

As consequence, in case of use of lambda based solution, user must decide if he want to pass each object as const and allow only calls on const object, by use of:

```
[s = Strategy{}] (auto&&... args) { return s.process(std::forward<decltype(args)>(args)...); };
```

Or allow modification of stored objects, but limits calls to non-const functors only:

```
[s = Strategy{}] (auto&&... args) mutable { return s.process(std::forward<decltype(args)>(args)...); };
```

Same problems may occurs in situation when stored function supports both const and mutable calls via appropriate overloads of `operator()`. For example in case of following class:

```
struct Mapper
{
  auto operator()(int i, int j) -> std::string& { return _mapping[{i, j}]; }
  auto operator()(int i, int j) const -> std::string const& { return _mapping[{i, j}]; }

private:
  std::map<std::pair<int, int>, std::string> _mapping;
};
```

Functors produced by `std::bind(Mapper{}, _1, 10)` and `bind_back(Mapper{}, 10)` will call both const and non-const overloads, depending on their qualification. While in case of lambda, user will need to decide to support only one of them, by using one of:

```
[m = Mapper{}](int i) -> std::string const& { return m(i, 10); }
[m = Mapper{}](int i) mutable -> std::string& { return m(i, 10); }
```

3.3. Preserving return type

The reader may notice that lambda functions used in previous section, are explicitly specifying their return type. This is caused by the fact that lambda is using the `auto` deduction for the return type as default. As consequence the following slightly changed declaration would return `std::string` object by value:

```
auto fc = [m = Mapper{}](int i) { return m(i, 10); };
auto fm = [m = Mapper{}](int i) mutable { return m(i, 10); };
```

Such slight change of code may lead to various changes in the behaviour of the program. Firstly additional copy construction will be invoked, if the object returned by the lambda is captured by reference:

```
auto const& s1 = fc(2);
auto const& s2 = fm(2);
```

Secondly, the lifetime of object returned from the functor will not longer be tied to the lifetime of the `Mapper` object, which may lead to creation of dangling references:

```
auto f = [m = Mapper{}](int i) { return m(i, 10); };
std::string* ps = nullptr;
{
  auto const& s = f(2);
  ps = &s;
}
// *ps is dangling
```

Lastly in case of the mutable version of functor, changing the result of the invocation would modifies temporary not mapped value:

```
fm(2) = "something";
```

To avoid such problems we may use `decltype`-based return type deduction, as it is done in case of `std::bind` and proposed `bind_front`/`bind_back`:

```
[m = Mapper{}](int i) -> decltype(auto) { return m(i, 10); }
```

3.4. Preserving value category

If we consider following example implementation of the functor that performs memoization of the expensive to compute function `func`:

```
struct CachedFunc
{
  std::string const& operator()(int i, int j) &
  {
     key_type key(i, j);

     auto it = _cache.find(key);
     if (it == _cache.end())
       it = _cache.emplace(std::move(key), func(i, j)).first;

     return it->second;
  }
```

```
private:
  using key_type = std::pair<int, int>;
  std::map<key_type, std::string> _cache;
};
```

As we can see `CachedFunc::operator()` is using reference qualification to limit valid calls only to lvalues. Use of this qualification allows us to avoid dangling reference problems, in situation when reference returned by temporary `CachedFunc` object would be used after its destruction. In addition it signals that use of `CachedFunc` makes sense only in situation when it is invoked multiple times and for one-shot invocation invoking `func` directly is more optimal solution.

As in case of the `const` propagation, we would like to preserve/propagate value category from the functor to stored callable. That means that for the following declarations:

```
auto f = [cache = CachedFunc{}] (int j) mutable -> std::string& { return cache(10, j); }; // 1
auto f = std::bind(CachedFunc{}, 10, _1); // 2
auto f = bind_front(CachedFunc{}, 10); // 3
```

Invocation on the lvalue (`f(1)`) shall perform call on the lvalue of `CachedFunc` and be well-formed, while invocation on the rvalue (`std::move(f)(1)`) shall lead to call on the rvalue and be ill-formed.

Out of discussed option, only proposed `bind_front`/`bind_back` (3) functions are preserving value category. In case of existing `std::bind` and lambda solutions, the call is always performed on lvalue regardless of the category of function object, and essentially bypass reference qualification.

Same problems also occurs in case of the bound arguments, even if the callable does not differentiate between calls on lvalues and rvalues. For example if we consider following function declarations

```
void foo(std::string&);
auto make_bind(std::string s)       { return std::bind(&foo, s); }
auto make_lambda(std::string s)     { return [s] { return foo(s); }; }
auto make_bind_front(std::string s) { return bind_front(&foo, s); }
auto make_bind_back(std::string s)  { return bind_back(&foo, s); }
```

Invocations in the form `make_bind("a")()` and `make_lambda("a")()` are well-formed and are invoking function `foo` with lvalue reference to temporary string. In case of proposed functions, value category of the functor also affects stored arguments and corresponding calls `make_bind_front("a")()` and `make_bind_back("a")()` are ill-formed.

3.5. Supporting one-shot invocation

Lack of propagation of the value category in existing partial function application solutions, prevents them from supporting functors that allows one-shot invocation via rvalue qualified call operator. As consequence for the following declarations:

```
struct CallableOnce
{
  void operator()(int) &&;
};

auto make_bind(int i)       { return std::bind(CallableOnce{}, i); }
auto make_lambda(int i)     { return [f = CallableOnce{}, i] { return f(i); }; }
auto make_bind_front(int i) { return bind_front(CallableOnce{}, i); }
auto make_bind_back(int i)  { return bind_back(CallableOnce{}, i); }
```

Only the invocation `make_bind_front(1)()` and `make_bind_back(1)()` are well formed, as the other two (`make_bind(1)()` and `make_lambda(1)()`) leads to unsupported call on the lvalue of `CallableOnce`.

It case of use of lambda expression it would be possible to workaround the problem by explicit use of the `std::move`:

```
[f = CallableOnce{}, i] { return std::move(f)(i); }
```

However above code is forcing calls on rvalue of `CallableOnce`, even if lvalue functor is invoked. As consequence multiple calls may be performed on single instance of `CallableOnce` class.

It is also worth to notice, that one-shot callable functors may also be produced as a result on binding an non-moveable type. For example in situation when we want to bind arguments to a function `consume` that accepts `std::unique_ptr<Obj>` by value:

```
struct ConsumeBinder
{
  ConsumeBinder(std::unique_ptr<Obj> p)
    : ptr(std::move(p))
  {}

  void operator()() &&
  { return cosume(std::move(ptr)); }

private:
  std::unique_ptr<Obj> ptr;
};
```

In addition support for one-shot invocation is leading to improved performance. For example let consider situation, when we want to bind a vector `v` as the first argument to the following function:

```
void bar(std::vector<int>, int)
```

Depending on the scenario, at the point of the call of the bind-wrapper (`bw`) that we will create, we may want to:

- move stored vector to `bar` function, if `bw` will be called only once (one-shot)
- copy stored vector to `bar` function, if `bw` will be called multiple times

Proposed `bind_front` function support both scenarios, via rvalue and lvalue overloads of call operator. Consequently if `bw` is created using `bind_front(&bar, v)`:

- `std::move(bw)(10)` will move stored vector (pass as rvalue reference)
- `bw(10)` will copy stored vector (pass as lvalue reference)

4. Design Decisions

The section provides rationale for deprecating existing `std::bind` even in the situation when proposed new functions does not strictly supersede its functionality.

4.1. No arbitrary argument rearrangements

In contrast to the `std::bind` proposed `bind_front`/`bind_back` does not support rearrangements or dropping of the call arguments, that was supported by `std::bind`. The reasoning behind this decision is twofold.

Firstly, handling of the placeholder was requiring a large amount of the meta-programing, to only determine types and values of the argument that will be actually passed to stored callable. However in this case required complexity of implementation is not only affecting the vendors, but also leads to unreadable error message produced to the user.

Secondly, repeated uses of the placeholder leads to double move of the passed object and as consequence unspecified values of the arguments. Occurrence of this problem depends both of bound arguments passed to the bind and once that are provided on the call side. This problem could potentially be fixed by introducing another type of placeholder that would pass rvalues as const rvalue reference or additional logic that will detect duplicated arguments on compile them and modify their value category. However both solutions would only increase the complexity of implementation and use.

4.2. No nested *bind expressions*

Both proposed function does not give provide any special meaning to the nested *bind expressions* (functors produced by `std::bind`) and their are passed directly to the stored callable in case of the invocation.

Firstly, in the author opinion, use of nested bind leads to unreadable code that are clearly improved by being replaced with custom functor, especially in situation when such functor can be created in place using lambda expression.

Secondly, special treatment of nested *bind expressions* and placeholders hardens the reasoning about behaviour of bind expression, by leading to the situations when `std::bind(f, a, b, c)()` is not invoking `f(a, b, c)`, despite the user intent. This may occur in situation when type of values passed to `std::bind` are not know by the programmer at point of binding:

```
struct apply_twice
{
  template<typename F, typename V>
  auto operator()(F const& f, V const& v) const
    -> decltype(f(f(v)))
  { return f(f(v)); }
};

template<typename F>
auto twicer(F&& f)
{ return std::bind(apply_twice{}, std::forward<F>(f), _1); }

double cust_sqrt(double x) { return std::sqrt(x); }
double cust_pow(double x, double n) { return std::pow(x, n); }
```

Invocation of `twicer(&cust_sqrt)(16)` is valid and return 2, while `twicer(std::bind(&cust_pow, _1, 2))(2))` is invalid.

4.3. Fixing `std::bind`

Additional motivation for introduction of then new function, is that fixing the problems mentioned above in `std::bind` would require introduction of breaking changes to the existing codebase. Furthermore such changes would not only take verbose form, when previously valid code will no longer compile, but may also silently change its meaning, by selecting different overload of underlining functor. The author believes that in such case introduction of new functions would be required anyway.

5. Impact On The Standard

This proposal has no dependencies beyond a C++14 compiler and Standard Library implementation.

Nothing depends on this proposal.

6. Proposed Wording

To be determined.

7. Implementability

Example implementation of proposed `bind_front`:

```
template<typename Func, typename BoundArgsTuple, typename... CallArgs>
decltype(auto) bind_front_caller(Func&& func, BoundArgsTuple&& boundArgsTuple, CallArgs&&... callArgs)
{
  return std::apply([&func, &callArgs...](auto&&... boundArgs)
         {
           return std::invoke(std::forward<Func>(func), std::forward<decltype(boundArgs)>(boundArgs)..., std::forward<CallArgs>(callArgs)...);
         }, std::forward<BoundArgsTuple>(boundArgsTuple));
}

template<typename Func, typename... BoundArgs>
class bind_front_t
{
public:
  template<typename F, typename... BA,
           std::enable_if_t<!(sizeof...(BA) == 0 && std::is_base_of_v<bind_front_t, std::decay_t<F>>), bool> = true>
  explicit bind_front_t(F&& f, BA&&... ba)
    : func(std::forward<F>(f))
    , boundArgs(std::forward<BA>(ba)...)
  {}

  template<typename... CallArgs>
  auto operator()(CallArgs&&... callArgs) &
    -> std::result_of_t<Func&(BoundArgs&..., CallArgs...)>
  { return bind_front_caller(func, boundArgs, std::forward<CallArgs>(callArgs)...); }

  template<typename... CallArgs>
  auto operator()(CallArgs&&... callArgs) const &
    -> std::result_of_t<Func const&(BoundArgs const&..., CallArgs...)>
  { return bind_front_caller(func, boundArgs, std::forward<CallArgs>(callArgs)...); }

  template<typename... CallArgs>
  auto operator()(CallArgs&&... callArgs) &&
    -> std::result_of_t<Func(BoundArgs..., CallArgs...)>
  { return bind_front_caller(std::move(func), std::move(boundArgs), std::forward<CallArgs>(callArgs)...); }

  template<typename... CallArgs>
  auto operator()(CallArgs&&... callArgs) const &&
    -> std::result_of_t<Func const(BoundArgs const..., CallArgs...)>
  { return bind_front_caller(std::move(func), std::move(boundArgs), std::forward<CallArgs>(callArgs)...); }

private:
  Func func;
  std::tuple<BoundArgs...> boundArgs;
};

template<typename Func, typename... BoundArgs>
auto bind_front(Func&& func, BoundArgs&&... boundArgs)
{
  return bind_front_t<std::decay_t<Func>, decay_unwrap_t<BoundArgs>...>{std::forward<Func>(func), std::forward<BoundArgs>(boundArgs)...};
}
```

To properly handle `std::reference_wrapper` in above code, we use `decay_unwrap` auxilary metafunction from [D0318R0: `decay_unwrap` and `unwrap_reference`] paper:

```
template<typename T>
struct decay_unwrap;

template<typename T>
struct decay_unwrap<std::reference_wrapper<T>>
{
  using type = T&;
};

template<typename T>
struct decay_unwrap
  : std::conditional_t<
      !std::is_same<std::decay_t<T>, T>::value,
      decay_unwrap<std::decay_t<T>>,
      std::decay<T>
   >
{};

template<typename T>
using decay_unwrap_t = typename decay_unwrap<T>::type;
```

8. Acknowledgements

Proposed runtime version of `bind_front` and `bind_back` are inspired by their compile time counterparts from Eric Niebler's Tiny Metaprogramming Library.

Special thanks and recognition goes to Sabre (http://www.sabre.com) for supporting the production of this proposal, and for sponsoring author's trip to the Oulu for WG21 meeting.

9. References

1. Chris Jefferson, Ville Voutilainen, "Bug 40 - variadic bind" (LEWG Bug 40, https://issues.isocpp.org/show_bug.cgi?id=40)
2. Mikhail Semenov, "Introducing an optional parameter for mem_fn, which allows to bind an object to its member function" (N3702, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3702.htm)
3. Tomasz Kamiński, "Parameter group placeholders for bind" (N4171, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4171.html)
4. Vicente J. Botet Escribá, "`decay_unwrap` and `unwrap_reference`" (D0318R0, https://github.com/viboes/std-make/blob/master/doc/proposal/utilities/p0318r0.pdf)